JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class OptionalInt

java.lang.Object
    java.util.OptionalInt

```
public final class OptionalInt
extends Object
```

A container object which may or may not contain an `int` value. If a value is present, `isPresent()` returns `true`. If no value is present, the object is considered *empty* and `isPresent()` returns `false`.

Additional methods that depend on the presence or absence of a contained value are provided, such as `orElse()` (returns a default value if no value is present) and `ifPresent()` (performs an action if a value is present).

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

API Note:

`OptionalInt` is primarily intended for use as a method return type where there is a clear need to represent "no result." A variable whose type is `OptionalInt` should never itself be `null`; it should always point to an `OptionalInt` instance.

Since:

1.8

Method Summary

All Methods	Static Methods	
Modifier and Type	**Method**	**Description**
`static OptionalInt`	**`empty`**`()`	Returns an empty `OptionalInt` instance.
`boolean`	**`equals`**`(Object obj)`	Indicates whether some other object is "equal to" this `OptionalInt`.
`int`	**`getAsInt`**`()`	If a value is present, returns the value, otherwise throws `NoSuchElementException`.
`int`	**`hashCode`**`()`	

		Returns the hash code of the value, if present, otherwise `0` (zero) if no value is present.
void	**ifPresent**(IntConsumer action)	If a value is present, performs the given action with the value, otherwise does nothing.
void	**ifPresentOrElse**(IntConsumer action, Runnable emptyAction)	If a value is present, performs the given action with the value, otherwise performs the given empty-based action.
boolean	**isEmpty**()	If a value is not present, returns `true`, otherwise `false`.
boolean	**isPresent**()	If a value is present, returns `true`, otherwise `false`.
static OptionalInt	**of**(int value)	Returns an `OptionalInt` describing the given value.
int	**orElse**(int other)	If a value is present, returns the value, otherwise returns `other`.
int	**orElseGet**(IntSupplier supplier)	If a value is present, returns the value, otherwise returns the result produced by the supplying function.
int	**orElseThrow**()	If a value is present, returns the value, otherwise throws `NoSuchElementException`.
		If a value is present, returns the value,

<X extends Throwable> int	**orElseThrow**(Supplier<? extends X> exceptionSupplier)	otherwise throws an exception produced by the exception supplying function.
IntStream	**stream**()	If a value is present, returns a sequential IntStream containing only that value, otherwise returns an empty IntStream.
String	**toString**()	Returns a non-empty string representation of this OptionalInt suitable for debugging.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

empty

```
public static OptionalInt empty()
```

Returns an empty OptionalInt instance. No value is present for this OptionalInt.

API Note:

Though it may be tempting to do so, avoid testing if an object is empty by comparing with == or != against instances returned by OptionalInt.empty(). There is no guarantee that it is a singleton. Instead, use isEmpty() or isPresent().

Returns:

an empty OptionalInt

of

```
public static OptionalInt of(int value)
```

Returns an OptionalInt describing the given value.

Parameters:

`value` - the value to describe

Returns:

an `OptionalInt` with the value present

getAsInt

```
public int getAsInt()
```

If a value is present, returns the value, otherwise throws `NoSuchElementException`.

API Note:

The preferred alternative to this method is `orElseThrow()`.

Returns:

the value described by this `OptionalInt`

Throws:

`NoSuchElementException` - if no value is present

isPresent

```
public boolean isPresent()
```

If a value is present, returns `true`, otherwise `false`.

Returns:

`true` if a value is present, otherwise `false`

isEmpty

```
public boolean isEmpty()
```

If a value is not present, returns `true`, otherwise `false`.

Returns:

`true` if a value is not present, otherwise `false`

Since:

11

ifPresent

```
public void ifPresent(IntConsumer action)
```

If a value is present, performs the given action with the value, otherwise does nothing.

Parameters:

`action` - the action to be performed, if a value is present

Throws:

`NullPointerException` - if value is present and the given action is `null`

ifPresentOrElse

```
public void ifPresentOrElse(IntConsumer action,
                            Runnable emptyAction)
```

If a value is present, performs the given action with the value, otherwise performs the given empty-based action.

Parameters:

`action` - the action to be performed, if a value is present

`emptyAction` - the empty-based action to be performed, if no value is present

Throws:

`NullPointerException` - if a value is present and the given action is `null`, or no value is present and the given empty-based action is `null`.

Since:

9

stream

```
public IntStream stream()
```

If a value is present, returns a sequential `IntStream` containing only that value, otherwise returns an empty `IntStream`.

API Note:

This method can be used to transform a `Stream` of optional integers to an `IntStream` of present integers:

```
Stream<OptionalInt> os = ..
IntStream s = os.flatMapToInt(OptionalInt::stream)
```

Returns:

the optional value as an `IntStream`

Since:

9

orElse

```
public int orElse(int other)
```

If a value is present, returns the value, otherwise returns `other`.

Parameters:

`other` - the value to be returned, if no value is present

Returns:

the value, if present, otherwise `other`

orElseGet

```
public int orElseGet(IntSupplier supplier)
```

If a value is present, returns the value, otherwise returns the result produced by the supplying function.

Parameters:

`supplier` - the supplying function that produces a value to be returned

Returns:

the value, if present, otherwise the result produced by the supplying function

Throws:

`NullPointerException` - if no value is present and the supplying function is `null`

orElseThrow

```
public int orElseThrow()
```

If a value is present, returns the value, otherwise throws `NoSuchElementException`.

Returns:

the value described by this `OptionalInt`

Throws:

`NoSuchElementException` - if no value is present

Since:

10

orElseThrow

```
public <X extends Throwable> int orElseThrow
(Supplier<? extends X> exceptionSupplier)
                                        throws X
```

If a value is present, returns the value, otherwise throws an exception produced by the exception supplying function.

API Note:

A method reference to the exception constructor with an empty argument list can be used as the supplier. For example, `IllegalStateException::new`

Type Parameters:

`X` - Type of the exception to be thrown

Parameters:

`exceptionSupplier` - the supplying function that produces an exception to be thrown

Returns:

the value, if present

Throws:

`X` - if no value is present

`NullPointerException` - if no value is present and the exception supplying function is `null`

equals

```
public boolean equals(Object obj)
```

Indicates whether some other object is "equal to" this `OptionalInt`. The other object is considered equal if:

- it is also an `OptionalInt` and;
- both instances have no value present or;
- the present values are "equal to" each other via ==.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - an object to be tested for equality

Returns:

`true` if the other object is "equal to" this object otherwise `false`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns the hash code of the value, if present, otherwise `0` (zero) if no value is present.

Overrides:

hashCode in class Object

Returns:

hash code value of the present value or `0` if no value is present

See Also:

Object.equals(java.lang.Object),
System.identityHashCode(Object)

toString

```
public String toString()
```

Returns a non-empty string representation of this `OptionalInt` suitable for debugging. The exact presentation format is unspecified and may vary between implementations and versions.

Overrides:

toString in class Object

Implementation Requirements:

If a value is present the result must include its string representation in the result. Empty and present `OptionalInt`s must be unambiguously differentiable.

Returns:

the string representation of this instance

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.